

Основи на управлението на паметта

1. Какво е управление на паметта?

Управлението на паметта е наборът от механизми, чрез които операционната система (ОС) разпределя, защитава и следи използването на оперативната памет (RAM) от процесите.

Цели на управлението на паметта:

- Изоляция и защита между процесите
- Ефективно разпределяне и освобождаване на памет
- Илюзия за голямо и непрекъснато адресно пространство (виртуална памет)
- Висока производителност чрез кеширане (TLB) и добри алгоритми за подмяна

2. Адресно пространство: физическа vs виртуална памет

Физическа памет

- Реалната RAM в машината
- Ограничена по размер
- Достъпът е бърз, но ресурсът е скъп

Виртуална памет

- Логическо адресно пространство за всеки процес
- Предоставя непрекъснато пространство, макар физически да е разпокъсано
- Позволява използване на диск (swap) като разширение

Карта на адресите:

Програма генерира ВИРТУАЛЕН адрес → MMU (хардуер) → ФИЗИЧЕСКИ адрес в RAM

Ако страницата липсва в RAM → Page Fault → Зареждане от диск (swap/файл)

3. Стратегии за разпределяне на памет

Непрекъснато разпределяне

- Целият процес заема един голям блок
- Прости алгоритми: First Fit, Best Fit, Worst Fit
- Проблем: външна фрагментация

Сегментиране

- Паметта се разделя на логически сегменти (код, данни, стек)
- По-лесна защита на отделните сегменти
- Все още е възможна фрагментация

Страничност (Paging)

- Виртуалната памет се дели на страници; физическата – на рамки
- Премахва външната фрагментация
- Ниска вътрешна фрагментация (в края на страницата)

Алгоритми за избор на свободен блок (непрекъснато разпределяне):

- **First Fit** – избира първия достатъчно голям свободен блок
- **Best Fit** – избира най-малкия възможен блок (минимизира остатъка)
- **Worst Fit** – избира най-големия блок (оставя по-големи остатъци)

4. Страничност: таблици на страници и TLB

- **Таблица на страници** – за всеки процес съпоставя виртуални страници към физически рамки
- **MMU** – хардуерен модул, който превежда адресите
- **TLB (Translation Lookaside Buffer)** – бърз кеш на последните преводи

Поток на превод на адрес:

CPU → виртуален адрес → TLB? (ако hit → директен превод)
ако miss → чете таблица на страници → актуализира TLB → достъп до RAM

5. Подмяна на страници и липси (Page Fault)

Кога възниква Page Fault?

- Страницата не е в RAM в момента на достъп
- ОС трябва да я зареди от диск (бавна операция)

Алгоритми за подмяна

- **FIFO** - най-старата страница се изважда
- **LRU** - изважда най-отдавна неизползваната
- **Clock** - приблизителен LRU с бит за използване

Работа с активен набор (Working Set)

ОС се стреми да държи в RAM страниците, които процесът използва най-често в момента, за да намали броя на Page Fault-овете.

6. Фрагментация на паметта

Външна фрагментация

- Свободната памет е разпиляна на малки дупки
- Решения: страничност, компактуване (скъпо)

Вътрешна фрагментация

- Резервирано е повече от нужното (напр. цяла страница за малък блок)
- Намалява се чрез по-малки единици/алокатори

7. Защита и споделяне

- **Права за достъп:** Read, Write, Execute на ниво страница/сегмент
- **Изоляция:** процесите не могат да четат/пишат памет на други процеси
- **Споделени страници:** една и съща физическа рамка се картографира в няколко процеса (напр. споделени библиотеки)
- **Copy-on-Write (CoW):** копие се прави само при запис – спестява RAM

8. Swap (Размяна към диск)

Когато RAM не достига, ОС премества по-рядко използвани страници към диск (swap дял/файл). Това увеличава наличната виртуална памет, но е **многократно по-бавно** от RAM.

Практически ефект:

- При силен swap системата става мудна (много Page Fault-и)
- Решение: повече RAM, оптимизация на софтуера, настройка на swappiness

9. Примерен сценарий

- 1) Процес стартира и заявява 200 MB памет
- 2) ОС разпределя виртуални страници и ги картографира към физически рамки при нужда
- 3) Част от страниците се зареждат мързеливо (on-demand), при първи достъп
- 4) При недостиг на RAM → подмяна на страници (LRU/Clock)
- 5) Запис в споделена страница → Copy-on-Write → отделна физическа рамка

10. Резюме

- ОС дава илюзия за голяма, непрекъсната памет чрез **виртуална памет**
- **Страничност** премахва външната фрагментация и улеснява защита/споделяне
- **TLB** ускорява превода на адреси
- **Page Fault** е скъп – алгоритмите за подмяна са ключови
- **Copy-on-Write** и споделяне на страници пестят RAM